Assignment Beta – Agentic AI
Question 2

Sections 2-4

2. Experimental Comparison of Fine-Tuning Methods

2.1 Experimental Setup

The experiments used Qwen2.5-1.5B-Instruct as the base model. A task-specific e-commerce function-calling dataset containing 2,000 examples was prepared. The data was divided into 1,800 training examples and 200 test examples using a fixed split seed of 42. The examples covered four functions and were formatted so that the model generated JSON containing a function name and its arguments.

The assignment suggests using a function-calling dataset such as Glaive. However, the experiments reported here used a synthetic e-commerce dataset instead. It contained 500 examples for each of four functions, with five template variations per function. This is a methodological deviation from the suggested dataset and limits how broadly the results can be generalized. The results should therefore be interpreted as performance on this specific synthetic task rather than on a broad real-world function-calling benchmark.

The same train/test split, base model, prompt format, target modules (q_proj, k_proj, v_proj and o_proj), random seed, two training epochs, learning rate of 2×10^{-4}, and training schedule were used for the adapter comparisons. LoRA, QLoRA and DoRA used rank 8, with alpha 16 and dropout 0.05. QLoRA used 4-bit NF4 quantization, FP16 computation and double quantization. AdaLoRA was configured with an initial rank of 12 and target rank of 8.

2.2 Evaluation Metrics

Four measurements were used. Trainable parameters show how many model parameters were updated during fine-tuning. Peak GPU memory and training time indicate the resource cost. JSON validity measures whether the generated response can be parsed as JSON. Tool accuracy measures whether the predicted function name is correct. Full-match accuracy is stricter: both the function name and all arguments must match the expected output.

2.3 Results

Method	Trainable parameters	Trainable %	Train time (min)	Peak train memory (GB)	JSON valid	Tool accuracy	Full match
Base model	0	0%	—	—	100%	97%	45%
LoRA (FP16, r=8)	2,179,072	0.1410%	11.90	3.69	100%	100%	79%
QLoRA (NF4, r=8)	2,179,072	0.1410%	25.35	1.95	100%	100%	81%
DoRA (FP16, r=8)	2,279,424	0.1474%	17.83	5.05	100%	100%	81.5%
AdaLoRA (configured r12→r8)	3,270,064	0.2114%	19.32	3.79	100%	100%	75%

Note: Base-model peak memory during training was not recorded, so it is shown as unavailable. The base model's separately measured evaluation peak memory was 2.89 GB; this is not directly comparable to the training-memory measurements in the table.

All four fine-tuned methods produced valid JSON for all 200 test examples and achieved 100% tool accuracy. Their full-match scores differed because selecting the correct function does not guarantee that every argument is correct. DoRA achieved the highest full-match score at 81.5%, followed closely by QLoRA at 81%. LoRA reached 79%, while AdaLoRA reached 75% in this run.

QLoRA used the least training memory (1.95 GB), substantially below LoRA (3.69 GB) and DoRA (5.05 GB), but it took the longest to train (25.35 minutes). DoRA provided the best exact function-and-arguments result, but also had the highest recorded training memory among the adapter methods. These results show a trade-off between accuracy, memory use and training time.

Implementation limitation: AdaLoRA was instantiated with AdaLoraConfig and the requested initial and target ranks, but the training loop did not call the PEFT adaptive allocation update method (update_and_allocate). Consequently, the run does not establish a fully functioning adaptive-rank schedule. Its score is included as the result of the implemented configuration, but should not be treated as a definitive comparison of properly scheduled AdaLoRA.

3. Rank Experiment

3.1 Method

DoRA was selected for the rank experiment because it produced the highest full-match accuracy in the initial comparison. Three additional DoRA configurations were trained using ranks 4, 16 and 32. The remaining experimental setup was kept the same.

3.2 Results

DoRA rank	Trainable parameters	Trainable %	Training time (min)	Peak train memory (GB)	JSON valid	Tool accuracy	Full match
4	1,189,888	0.0770%	17.34	7.30	100%	100%	79%
16	4,458,496	0.2880%	17.17	7.35	100%	100%	80%
32	8,816,640	0.5679%	17.17	7.42	100%	100%	81%

Tool accuracy and JSON validity remained at 100% for all three ranks. Full-match accuracy increased from 79% at rank 4 to 80% at rank 16 and 81% at rank 32. However, trainable parameters increased from approximately 1.19 million to 8.82 million, while peak memory increased only slightly from 7.30 GB to 7.42 GB.

The results suggest that increasing rank provided a small improvement in exact argument matching, but the gain was modest compared with the increase in trainable parameters. Rank therefore helped slightly in this experiment, but higher rank did not produce a proportional improvement. The difference should also be interpreted cautiously because it comes from one run per rank.

4. Discussion and Production Recommendation

4.1 Production Recommendation

For this task, DoRA is the strongest choice when exact function-and-argument accuracy is the main priority: it achieved the highest full-match score (81.5%). However, its training memory requirement was the highest of the adapter methods at 5.05 GB. If deployment or training hardware has a strict memory limit, QLoRA is a practical alternative. It achieved 81% full-match accuracy while requiring only 1.95 GB of peak training memory. LoRA is also a reasonable simpler baseline, while the AdaLoRA result should be revisited with adaptive rank updates correctly enabled before making a production decision.

Inference speed was not directly benchmarked in these experiments, so no measured claim can be made that one method is faster at inference. In general, adapter choice, quantization, hardware and whether adapters are merged can affect serving latency. A production system should benchmark end-to-end latency under its expected concurrency and hardware before deployment.

4.2 Merging and Deployment Considerations

Standard LoRA represents an update using low-rank matrices that can generally be combined with the base model weights for deployment. QLoRA trains with a quantized base model; merging therefore requires care because the quantized weights may need to be dequantized for merging and then quantized again for a quantized deployment. This process can change numerical values and should be validated against the unmerged adapter.

DoRA separates weight magnitude and direction, so its weight reconstruction and merging procedure is not identical to ordinary LoRA. The supported PEFT merge path and the target deployment format should be tested rather than assuming that every LoRA merge workflow applies unchanged. In all cases, compare merged and unmerged outputs on a validation set before release.

4.3 Error Analysis: Illustrative Cases

The following are illustrative examples of plausible function-calling errors. They were not retrieved from the model's saved evaluation outputs and must not be interpreted as verified DoRA predictions. They demonstrate the kinds of mistakes that a qualitative error analysis should inspect.

Case	Expected behaviour	Illustrative incorrect behaviour	Error category
1	search_products with category "shoes"	search_products with category "sneakers"	Incorrect argument value
2	check_order with ID "ORD1024"	check_order with ID "ORD1042"	Incorrect identifier
3	track_shipment with tracking ID "TRK5678"	track_shipment with tracking ID omitted	Missing argument
4	search_products with maximum price 5000	search_products with maximum price 500	Incorrect numerical value
5	check_order with the required order ID	track_shipment called instead	Incorrect function selection

These examples highlight why JSON validity and tool accuracy are not sufficient on their own. A response may be valid JSON and select the correct tool while still containing an incorrect, missing or incomplete argument. Full-match accuracy captures these stricter requirements.

4.4 LoRA in Multi-Agent Systems

LoRA adapters can support specialization in multi-agent systems. For example, a shopping assistant could use separate adapters for product discovery, order tracking and customer support while sharing one base model. This can reduce the need to fine-tune and store a separate full model for every agent. Adapters can also be selected for different tasks as requests are routed through the system.

Live adapter switching introduces operational risks. A request could be routed to the wrong adapter, or concurrent requests could interfere if adapter state is shared unsafely. The system should associate each request with the correct adapter, isolate concurrent execution where necessary, and test routing behaviour. Incorrect adapter selection could lead to incorrect tool calls even when each adapter performs well on its own task.

4.5 What Was Most Surprising

The most notable result was that QLoRA achieved nearly the same full-match accuracy as DoRA while using considerably less training memory. DoRA achieved the highest full-match score, but its memory cost was higher. The rank experiment was also interesting: increasing DoRA rank from 4 to 32 improved full-match accuracy by only two percentage points, despite a substantial increase in trainable parameters. This suggests that a larger adapter is not automatically much better and that the best configuration depends on the task and available hardware.

Limitations

The dataset was synthetic and template-based rather than the suggested Glaive dataset, so the results may not represent performance on varied real-world user requests. The test set contained 200 examples, and the rank comparison used one run per rank. AdaLoRA adaptive allocation updates were not invoked in the implemented training loop. Inference speed was not directly measured. Finally, the five error cases above are illustrative rather than verified model outputs. These limitations should be considered when interpreting the results.